

Coding & Solution

Date	01 JULY 2026
Team ID	XXXXXX
Project Name	AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/rajeshkatamsetti-4546/AI-Powered-Debt-Relief-Financial-Recovery-Platform-Project.git
Programming Language(s)	Python
Key Features Implemented	User Authentication, Loan Management (CRUD), Financial Health Calculator, Debt Settlement Prediction, AI-Based Negotiation Support, SQLite Database Integration, REST API with Swagger Documentation, Interactive Streamlit Dashboard
Pending / Incomplete Features	Email Notifications, Cloud Deployment, Advanced AI Recommendation Engine, Multi-language Support
Setup / Run Instructions	1. Clone the repository. 2. Create and activate a virtual environment. 3. Install dependencies using pip install -r requirements.txt. 4. Start the backend using python -m uvicorn app.main:app --reload. 5. Start the frontend using streamlit run frontend/app.py. 6. Open the Streamlit application in your browser.
Key Features Implemented	User Authentication, Loan Management (CRUD), Financial Health Calculator, Debt Settlement Prediction, AI-Based Negotiation Support, SQLite Database Integration, REST API with Swagger Documentation, Interactive Streamlit Dashboard

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The project follows a modular architecture with separate backend (FastAPI) and frontend (Streamlit) components. It provides financial health analysis, loan management, debt settlement prediction, and AI-based negotiation support. The application is integrated with a SQLite database, includes Swagger API documentation, and runs successfully with a user-friendly interface. The project is ready for demonstration and future enhancements.